

High Level Design Finance Quiz / Tutor Bot

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrollment Number
01	Mahima Patidar	EN22CS301569
02	Nikhil Rathore	EN22CS301650
03	Nitisha Kushwah	EN22CS301664
04	Nidhish Mehta	EN22CS301642
05	Moin Husain	EN22IT301612

Group Name: D6 G12

Project Number: GAI-23

Industry Mentor Name:

University Mentor Name: Prof.Ajaj Khan

Academic Year: 2025-2026

1. Introduction

1.1 Scope of the Document

This document presents the High-Level Design (HLD) of a **Finance Quiz and Tutor Bot powered by Generative AI**, which is designed to deliver interactive and personalized financial education. The scope of this system includes the design of architecture, identification of major components, and explanation of how these components interact to provide a seamless user experience.

The system focuses on automating quiz generation, evaluating user responses, and providing intelligent explanations using AI. It also ensures scalability, security, and efficient deployment using modern cloud and DevOps practices. The document serves as a blueprint for implementing an AI-driven educational platform that can adapt to different user needs and learning levels.

1.2 Intended Audience

Project Supervisors

This section of the document is intended for supervisors to review the overall system design, assess the feasibility of the solution, and ensure that the project meets academic and technical objectives. It also helps them evaluate innovation in using Generative AI for education.

AI/ML Engineers

AI/ML engineers can use this document to understand how the Generative AI model is integrated into the system, how it generates quiz content, and how it processes user inputs to provide intelligent responses.

System Architects

System architects will analyze the structural design of the system, including component interactions, scalability considerations, and integration with cloud platforms and APIs.

1.3 System Overview

The Finance Quiz / Tutor Bot is an intelligent learning system that uses Generative AI to provide finance-related education in an interactive and engaging manner. The system allows users to take quizzes, receive instant feedback, and ask questions related to finance topics such as savings, investments, taxation, and banking.

The system integrates multiple technologies, including AI models for content generation, backend services for processing logic, and cloud infrastructure for scalability. It dynamically generates quiz questions instead of relying on static databases, ensuring variety and adaptability.

2. System Design

2.1 Application Design

The application follows a **modular and layered architecture**, which allows different components of the system to function independently while maintaining seamless communication. This design enhances maintainability and scalability.

- **ApplicationLayer**
This layer includes the user-facing components such as the quiz interface and chatbot. It is responsible for capturing user inputs and displaying results, feedback, and explanations in an intuitive format.
- **AILayer**
This is the core of the system where the Generative AI model processes user queries and generates quiz questions, answers, and explanations. It ensures that responses are context-aware and educational.
- **BackendLayer**
The backend manages business logic, processes API requests, and coordinates communication between the frontend and AI services. It also handles evaluation of quiz answers and stores user data.
- **DeploymentLayer**
This layer includes cloud services and CI/CD pipelines that automate deployment, scaling, and updates of the application.

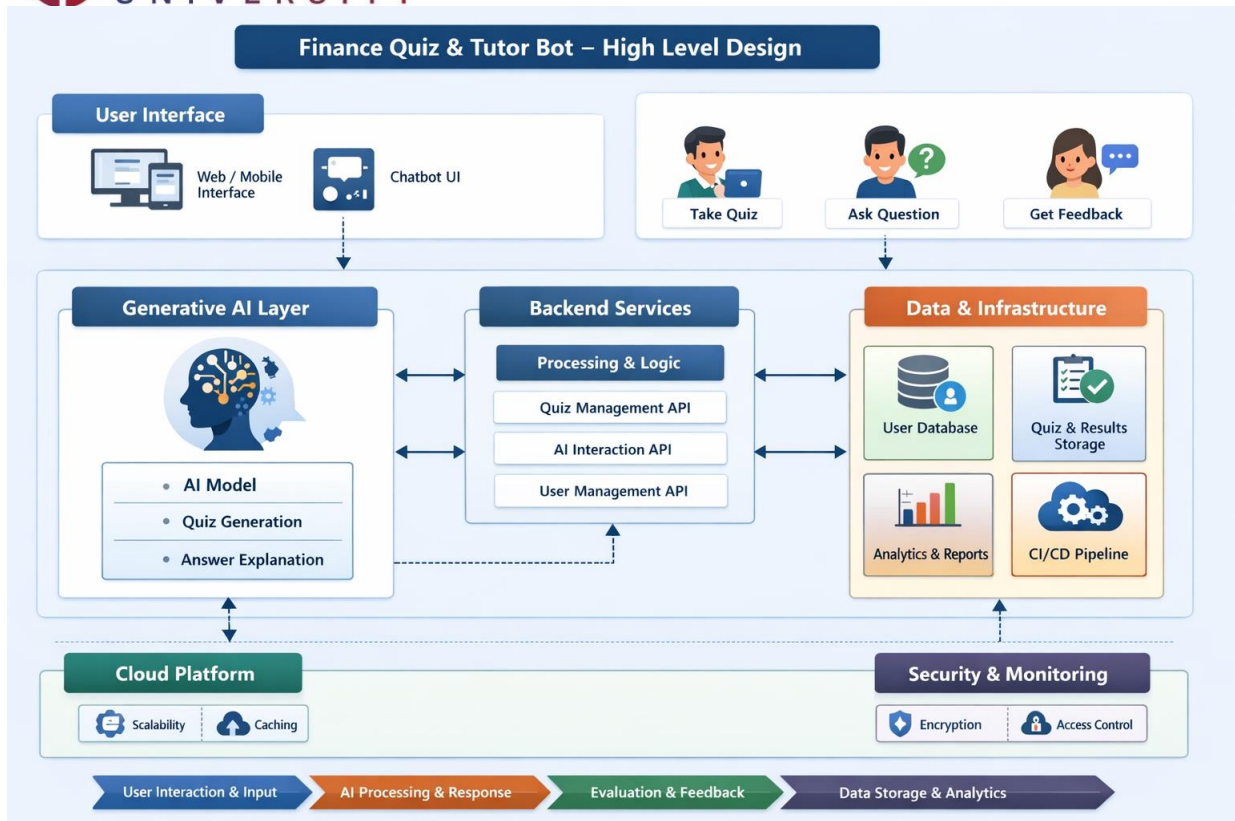
This structured approach ensures flexibility, making it easy to update or replace individual components without affecting the entire system.

2.2 Process Flow

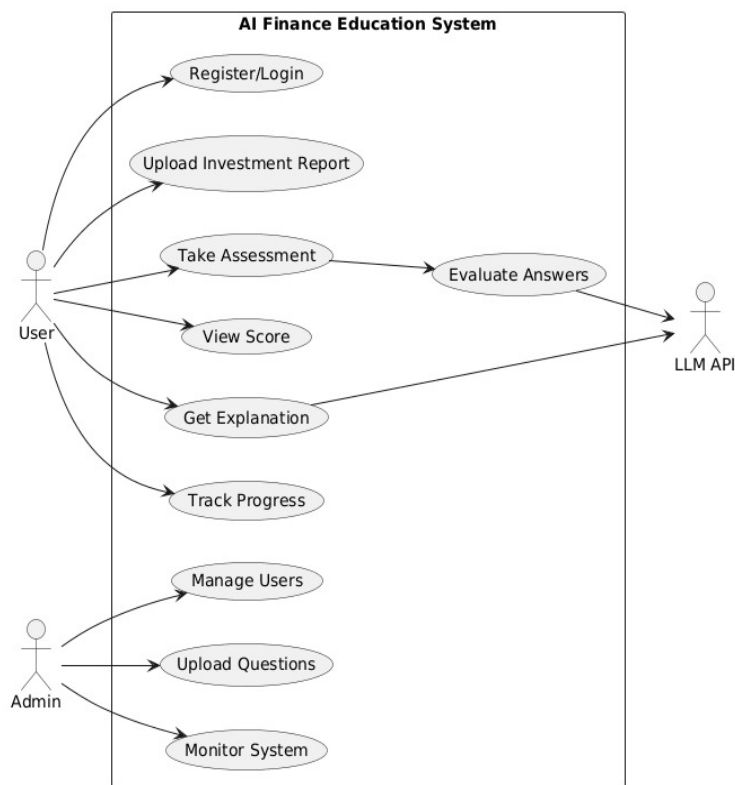
The process flow defines how the system operates from user interaction to response delivery.

1. The user logs into the system and selects a quiz or asks a question.
2. The request is sent from the frontend to the backend via APIs.
3. The backend forwards the request to the Generative AI model.
4. The AI model processes the request and generates a response (question, answer, or explanation).
5. If it is a quiz, the user submits answers which are evaluated by the backend.
6. The system generates feedback, including correct answers and explanations.
7. Results are displayed to the user and stored in the database.
8. The system updates user performance data for future personalization.

This automated workflow ensures real-time interaction and minimal delay.



High Level Articehcture



Usecase Diagram

2.3 Information Flow

The information flow begins when the user interacts with the system through the interface. The input data is transmitted to the backend server, where it is processed and forwarded to the AI model.

The AI model generates meaningful outputs such as quiz questions or explanations. These outputs are sent back to the backend, which formats them and sends them to the user interface.

Simultaneously, user interaction data such as answers, scores, and activity logs are stored in the database. This data is later used for analytics, performance tracking, and improving personalization.

2.4 Components Design

User Interface (UI)

The UI is responsible for providing a user-friendly platform where users can attempt quizzes and interact with the tutor bot. It includes dashboards, chat interfaces, and result displays.

Backend Server

Acts as the central processing unit of the system. It handles API requests, processes logic, evaluates answers, and communicates with the AI model and database.

Generative AI Model

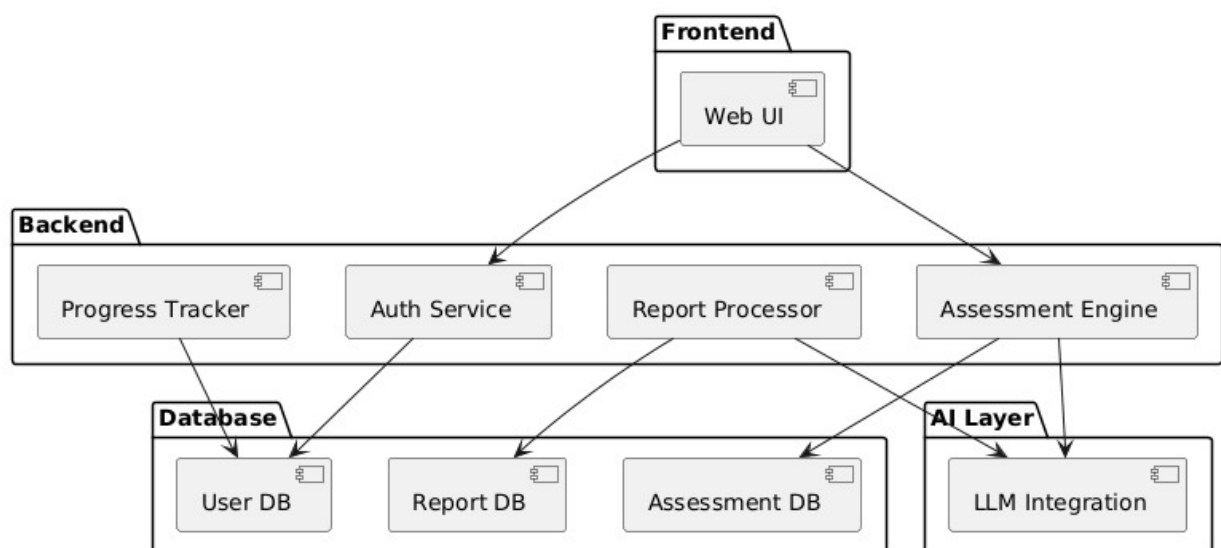
This component generates quiz questions, answers, hints, and explanations dynamically. It ensures that the content is relevant, accurate, and tailored to the user's level.

Database

Stores user profiles, quiz history, performance metrics, and system logs. It plays a crucial role in personalization and analytics.

CI/CD Pipeline

Automates building, testing, and deployment of the application, ensuring faster updates and reduced manual errors.



2.5 Key Design Considerations (Detailed)

Modularity

The system is designed using a modular architecture where each component operates independently and performs a specific function. For example, the user interface, backend logic, AI model, and database are all separated into distinct modules. This separation ensures that changes or updates in one module (such as upgrading the AI model or modifying the UI) do not affect the functioning of other components.

Scalability

Scalability is a critical aspect of the system, as the number of users interacting with the quiz bot may increase over time. The architecture is designed to handle both vertical scaling (increasing server capacity) and horizontal scaling (adding more servers).

Reliability

Reliability ensures that the system remains available and functional at all times. The system is designed with fault-tolerant mechanisms such as backup servers, retry mechanisms, and error-handling strategies.

Automation

Automation plays a key role in reducing manual effort and improving system efficiency. The entire lifecycle of the application—from code integration to deployment—is automated using CI/CD pipelines.

2.6 API Catalogue (Detailed)

Quiz Management API

This API is responsible for handling all quiz-related operations. It allows the system to generate quizzes dynamically, fetch questions, submit user answers, and return evaluation results. It ensures smooth interaction between the frontend and backend during quiz sessions.

AI Interaction API

This API acts as a bridge between the backend server and the Generative AI model. It sends user queries to the AI model and retrieves generated responses such as questions, explanations, and hints. It ensures that responses are delivered in real time with minimal latency.

User Management API

This API manages user-related operations such as registration, login, profile management, and authentication. It ensures secure access to the system and maintains user-specific data such as quiz history and performance.

Analytics API

This API collects and processes user performance data. It provides insights such as accuracy, progress, and improvement trends. These analytics are used to personalize the learning experience and improve system recommendations.

3. Data Design

3.1 Data Model

The data model defines how information is structured and stored within the system. It includes multiple entities such as users, quizzes, questions, answers, and performance metrics.

Each entity is designed to store specific types of information. For example, the user entity stores personal details and login credentials, while the quiz entity stores generated questions and their difficulty levels. The performance entity tracks scores, accuracy, and progress over time.

3.2 Logical Relationships

- **User → Quiz (1:N)**
A single user can attempt multiple quizzes over time, allowing continuous learning and progress tracking.
 - **Quiz → Questions (1:N)**
Each quiz consists of multiple questions generated dynamically by the AI model.
 - **User → Result (1:1 per attempt)**
Each quiz attempt generates a result that includes score, accuracy, and feedback.
 - **AI Model → Responses (1:N)**
The AI model generates multiple outputs such as questions, hints, and explanations for different user inputs.
 - **User → Performance Data (1:N)**
A user can have multiple performance records, which are used for analysis and personalization.
-

3.3 Data Access Mechanism

The system uses secure APIs and database queries to access and manage data. Authentication mechanisms such as tokens ensure that only authorized users can access sensitive data.

Role-based access control (RBAC) is implemented to restrict actions based on user roles (e.g., admin vs. student). Data encryption is used to protect sensitive information such as passwords and personal details.

3.4 Data Retention Policies

The system maintains historical data such as quiz attempts, scores, and interaction logs for analysis and auditing purposes.

User performance data is retained to provide personalized recommendations and track progress over time. Logs are stored for debugging and system monitoring. Old or unnecessary data may be archived or deleted periodically to optimize storage.

3.5 Data Migration

The system supports seamless data migration across different environments such as development, testing, and production.

Cloud-based infrastructure allows easy transfer of data and configurations. Backup and restore mechanisms ensure that no data is lost during migration. This capability is essential for scaling the system and maintaining consistency across environments.

4. Interfaces

4.1 User Interface

The user interface is designed to be intuitive and user-friendly, allowing users to easily interact with the quiz bot. It includes features such as quiz dashboards, chat windows, progress reports, and result displays.

The UI ensures smooth navigation and provides real-time feedback, enhancing the overall user experience.

Command Line Interface (CLI)

The CLI is used by developers and administrators to interact with the system. It allows them to deploy applications, manage configurations, and monitor system performance.

The CLI provides flexibility and control, especially during development and troubleshooting.

5. State and Session Management

The system maintains user sessions to track ongoing activities such as quiz attempts and chatbot conversations.

Session management ensures that users can continue their interaction without losing progress. For example, if a user disconnects temporarily, they can resume their quiz from where they left off.

State management also tracks system data such as AI responses and user inputs, ensuring consistency between different components of the system.

6. Caching (Detailed)

Caching is implemented to improve system performance and reduce response time. Frequently accessed data such as commonly asked questions, AI-generated responses, and static content are stored in cache memory.

This reduces the need to repeatedly generate the same responses using the AI model, saving computational resources and improving efficiency.

Caching mechanisms also help in reducing latency, ensuring faster user interactions and a smoother experience.

7. Functional Requirements

7.1 User Authentication & Authorization

The system must provide secure authentication mechanisms such as username-password login or OAuth-based login.

Authorization ensures that users can only access features relevant to their roles. For example, administrators can manage system settings, while users can only attempt quizzes and view results.

7.2 Quiz Management

The system should dynamically generate quizzes using the AI model based on selected topics and difficulty levels.

It should allow users to attempt quizzes, submit answers, and receive instant evaluation and feedback.

7.3 Performance Tracking

The system tracks user performance metrics such as scores, accuracy, and completion time.

This data is used to generate progress reports and provide personalized recommendations for improvement.

7.4 AI-Based Tutoring

The system acts as a virtual tutor by providing explanations, hints, and guidance.

It uses Generative AI to answer user queries and explain financial concepts in a simple and understandable manner.

7.5 Content Generation (Gen AI)

The system uses Generative AI to create quiz questions, answers, and explanations dynamically.

This ensures that content is always fresh, relevant, and tailored to the user's level of understanding.

7.6 CI/CD Workflow Configuration

The system automates the build, test, and deployment process using CI/CD pipelines.

This ensures that updates are delivered quickly and reliably without manual intervention.

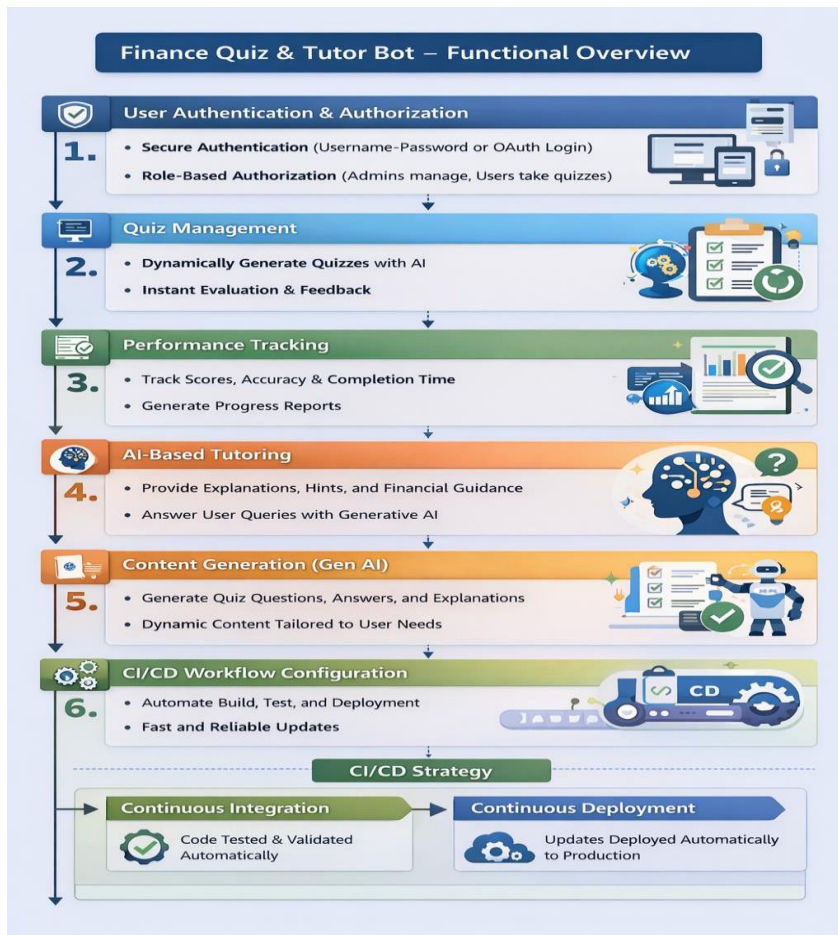
7.7 CI/CD Strategy

Continuous Integration

All code changes are automatically tested and validated before being merged into the main system. This ensures code quality and prevents errors.

Continuous Deployment

After successful testing, updates are automatically deployed to the production environment. This ensures that users always have access to the latest features without downtime.



8. Non-Functional Requirements

8.1 Security Aspects

The system ensures data security through encryption, secure APIs, and authentication mechanisms. Sensitive data such as user credentials and financial information are protected against unauthorized access.

Regular security audits and updates are performed to maintain system integrity.

8.2 Performance Aspects

The system is designed to provide fast and efficient responses, even under high load conditions.

Optimizations such as caching, load balancing, and efficient API design ensure minimal latency. The system can handle multiple concurrent users without performance degradation, ensuring a smooth and responsive user experience.

9. References

Problem Statement: **Finance Quiz/Tutor bot**

Project Group: **D6 – Group 12**

Project Number: **GAI-23**

10. Github

https://github.com/Mahi3343/finance_bot